



Recibe una cálida:

¡Bienvenida!

Te estábamos esperando 😊 +

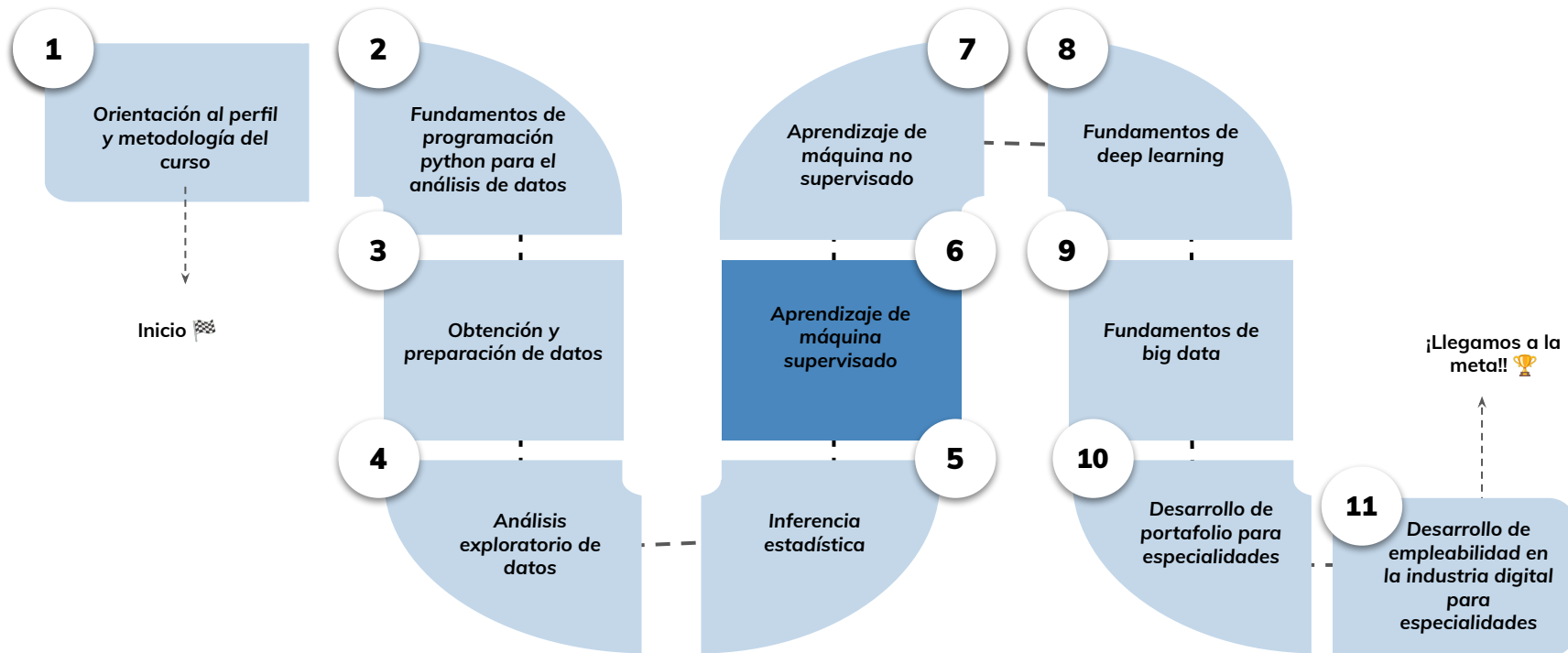


› Preprocesamiento y escalamiento de dato- Parte II

Aprendizaje Esperado 3: Aplicar técnicas para el preprocesamiento de datos en un modelamiento de aprendizaje de máquina utilizando las herramientas de python.

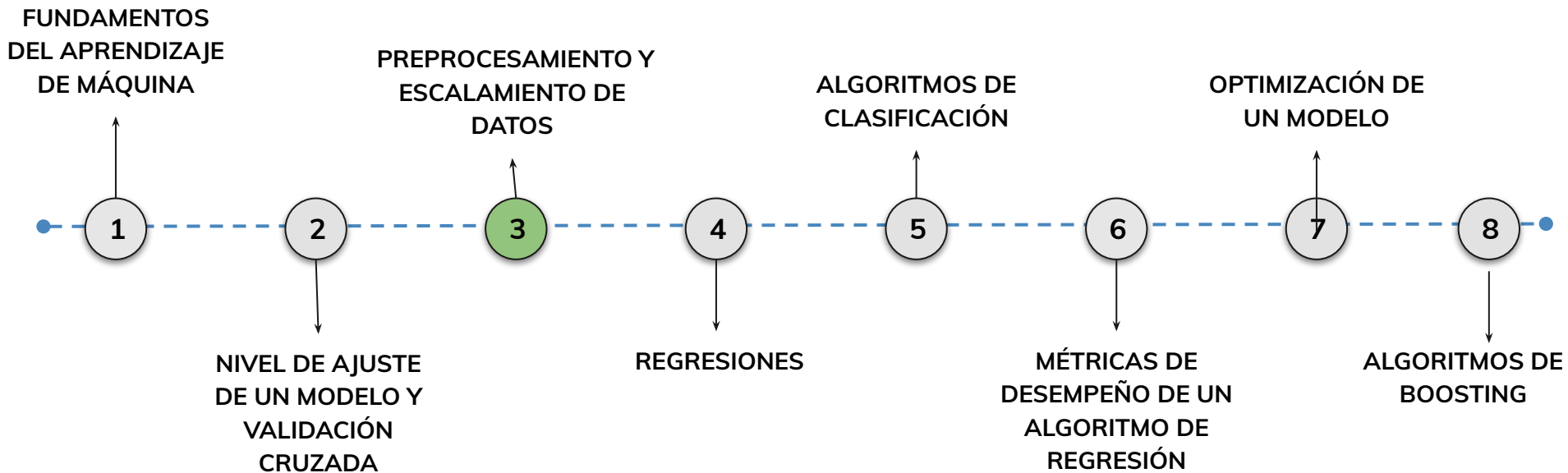
Hoja de ruta

¿Cuáles skills conforman el programa? **Fundamentos de Ciencia de Datos**



Roadmap de lecciones

¿Cuáles **lecciones** estaremos estudiando en este módulo?



Learning Path

¿Cuáles temas trabajaremos hoy?

3.

Preprocesamiento Avanzado

Consolidaremos herramientas para automatizar tareas comunes de preprocesamiento y dejar el dataset completamente listo para modelar.

Pipelines y
ColumnTransformer

Distancias y escalamiento

Estructura y
utilidad de los
pipelines

Transformaciones
por columna

Distancia
Euclideana vs
Manhattan

MinMaxScaler vs
StandardScaler y
cuándo usar cada uno



Objetivos de aprendizaje

¿Qué aprenderás?



- Implementar pipelines de preprocesamiento con Scikit-Learn
- Utilizar ColumnTransformer para aplicar transformaciones diferenciadas
- Calcular distancias (Euclideana y Manhattan) y entender su impacto
- Aplicar técnicas de escalamiento Min-Max y Z-Score
- Evaluar cuándo y por qué escalar variables

Repaso clase anterior

¿Quedó alguna duda?

En la clase anterior trabajamos :

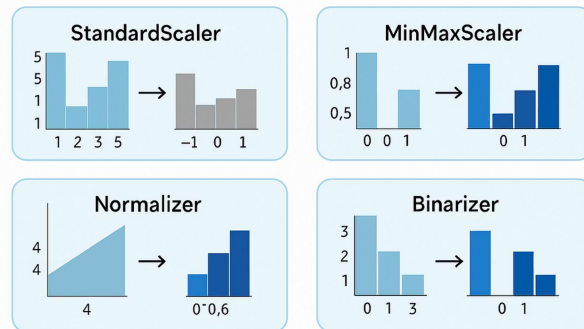
- Por qué es fundamental el preprocesamiento
- Cómo gestionar valores nulos y duplicados
- Imputación con SimpleImputer
- Codificación de variables categóricas con One-Hot y LabelEncode

Preprocesamiento y escalamiento de datos

Implementación de Preprocesamiento con Scikit-Learn

La librería **Scikit-Learn** es una de las herramientas más utilizadas en el ecosistema Python para la construcción de modelos de aprendizaje automático. Además de ofrecer una amplia gama de algoritmos, incluye múltiples utilidades para el preprocesamiento de datos, facilitando la transformación y preparación de los datos antes de entrenar un modelo.

scikit-learn Preprocessing Tools



Herramientas de Codificación en Scikit-Learn

LabelEncoder

Una de las funciones más utilizadas para transformar variables categóricas es LabelEncoder, que convierte cada categoría en un número entero. Este método es adecuado cuando la variable categórica tiene un orden implícito, pero puede ser problemático para variables sin orden, ya que el modelo podría interpretar los números como jerárquicos.

OneHotEncoder

Scikit-Learn también ofrece la clase OneHotEncoder, que transforma las variables categóricas en un array binario utilizando la técnica de codificación One-Hot. A diferencia de LabelEncoder, esta técnica es útil cuando no existe un orden entre las categorías y se desea evitar introducir un sesgo artificial.

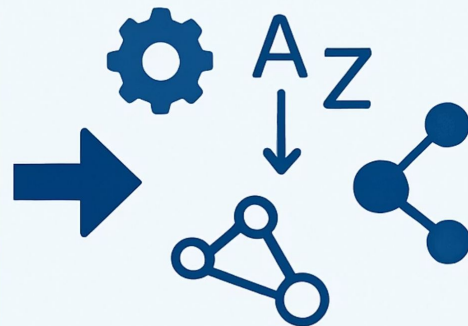
Integración con Pandas

Para trabajar con variables categóricas en un DataFrame de Pandas, es común combinar las herramientas de Scikit-Learn con las funciones de Pandas, como `pd.get_dummies()`, que permite generar variables dummy de manera rápida y eficiente. Esto resulta especialmente útil cuando se requiere eliminar la primera columna para evitar multicolinealidad.

! pandas

scikit *learn*

✓		
	✓	



PREPROCESSING

MISSING VALUES IMPUTATION WITH SIMPLEIMPUTER

1.0	2.0
5.0	NaN
4.0	3.0
NaN	6.0
3.3	4.0

SimpleImputer

1.0	2.0
5.0	3.7
4.0	1.0
4.0	2.0
3.3	4.0

Gestión de Valores Faltantes

Otra herramienta fundamental de Scikit-Learn es SimpleImputer, que permite gestionar los valores faltantes. Este transformador facilita la imputación de datos utilizando estrategias como la media, mediana o la moda, asegurando que los datos estén completos antes de entrenar el modelo.

Pipelines de Preprocesamiento

Creación del pipeline

Scikit-Learn incluye el módulo `sklearn.pipeline`, que permite encadenar varias transformaciones en un solo flujo de trabajo.

Definición de transformaciones

Esto es particularmente útil cuando se combinan varias tareas de preprocesamiento, como imputación, codificación y escalamiento, en un solo paso.

Aplicación del pipeline

La ejecución del pipeline garantiza que todas las transformaciones se apliquen en el orden correcto y de manera consistente.

Transformadores Avanzados

Además, la librería ofrece transformadores avanzados, como ColumnTransformer, que permite aplicar diferentes transformaciones a columnas específicas. Por ejemplo, se puede imputar los valores faltantes en columnas numéricas y aplicar One-Hot Encoding a las columnas categóricas dentro de un mismo pipeline.

Finalmente, la integración de Scikit-Learn con Pandas y NumPy permite realizar todas las tareas de preprocesamiento de manera eficiente y reproducible, facilitando la preparación de los datos y la construcción de modelos de alto rendimiento.

```
# Definir columnas
columnas_num = ['Edad', 'Salario']
columnas_cat = ['Ciudad']

# Definir transformadores
transformador_num = Pipeline(steps=[
    ('imputacion', SimpleImputer(strategy='mean')),
    ('escalado', StandardScaler())
])

transformador_cat = Pipeline(steps=[
    ('onehot', OneHotEncoder(drop='first'))
])
```

```
# Aplicar ColumnTransformer
preprocesador = ColumnTransformer(transformers=[
    ('num', transformador_num, columnas_num),
    ('cat', transformador_cat, columnas_cat)
])

# Transformar los datos
datos_procesados = preprocesador.fit_transform(datos)

print(datos_procesados)
```

Salida:

```
[[-1.34164079 -1.22474487  0.  1.]
 [ 1.34164079  1.22474487  0.  0.]
 [-0.4472136  0.  0.  1.]
 [ 0.4472136  0.  1.  0.]]
```

Ejemplo Práctico de Preprocesamiento con Scikit-Learn

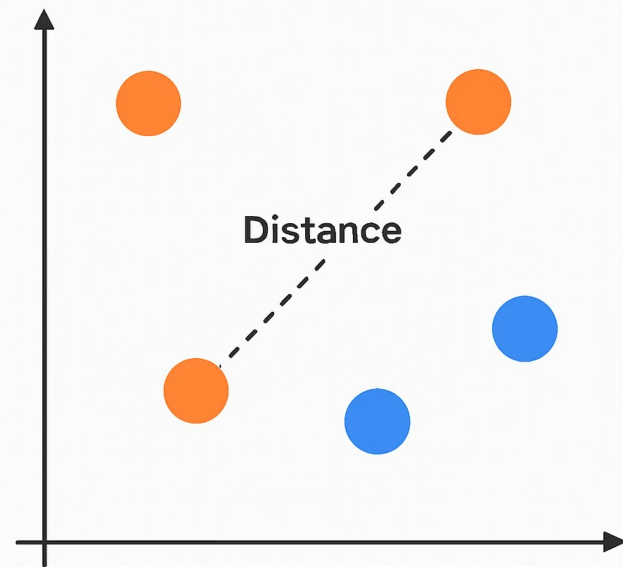
Consideremos el siguiente conjunto de datos:

ID	Edad	Ciudad	Salario
1	25	Madrid	30000
2	45	Sevilla	50000
3	30	Madrid	NaN
4	40	Barcelona	40000

El Concepto de Distancia

El concepto de **distancia** es fundamental en muchos algoritmos de aprendizaje automático, especialmente en aquellos basados en la similitud entre observaciones. La distancia cuantifica el grado de diferencia o similitud entre dos puntos en un espacio de características. Cuanto menor sea la distancia, mayor será la similitud entre los datos.

Distance in Machine Learning



Aplicaciones de la Distancia en Machine Learning



K-Nearest Neighbors

Algoritmo de clasificación y regresión que utiliza la distancia para encontrar los k ejemplos más cercanos.



K-Means Clustering

Algoritmo de agrupamiento que asigna observaciones a clusters basándose en la distancia a los centroides.



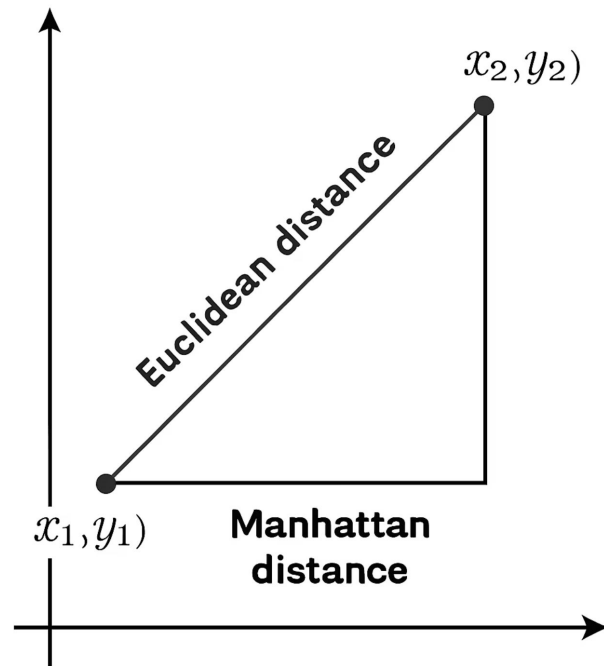
Reducción de dimensionalidad

Técnicas como t-SNE que preservan las distancias entre puntos al proyectarlos en espacios de menor dimensión.

Métricas de Distancia

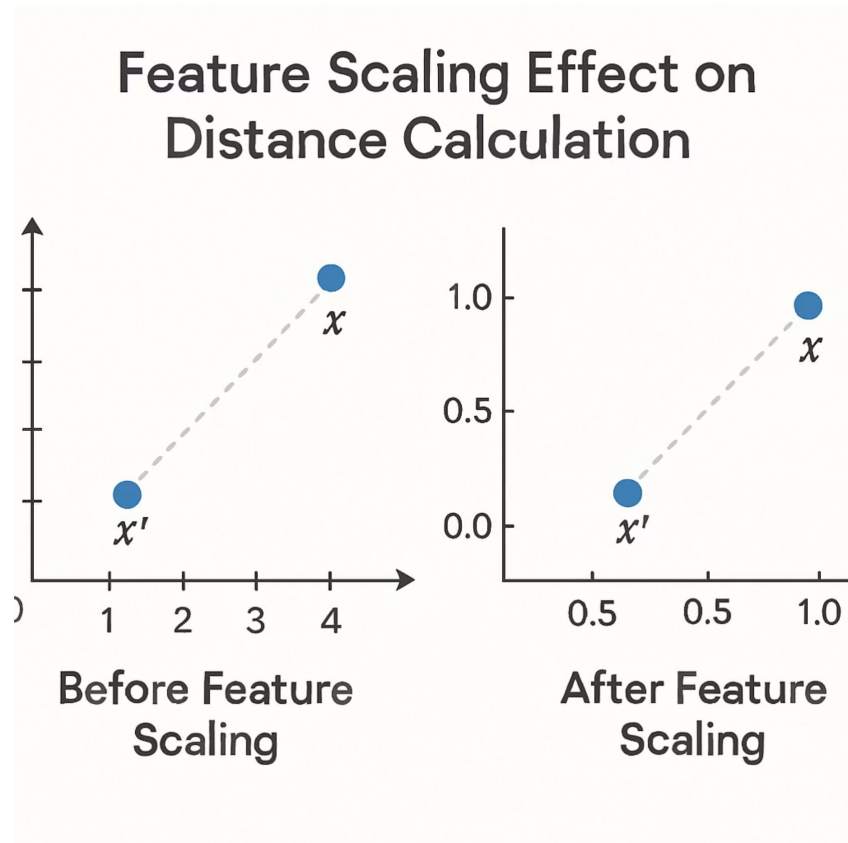
Existen diferentes formas de calcular la distancia entre dos puntos, y la elección de la métrica adecuada depende de la naturaleza de los datos y del problema específico. La más utilizada es la **distancia euclídeana**, que mide la distancia "en línea recta" entre dos puntos en un espacio n-dimensional.

Otra métrica común es la **distancia de Manhattan**, también conocida como distancia del taxi, que mide la distancia total recorrida a lo largo de los ejes del espacio. Esta métrica es útil cuando las diferencias absolutas entre características son más relevantes que la distancia en línea recta.



Importancia del Escalamiento para la Distancia

El cálculo de la distancia requiere que todas las características numéricas estén en la misma escala. Si una característica tiene valores mucho mayores que otra, su influencia dominará el cálculo de la distancia. Por ello, el preprocesamiento y el escalamiento de datos son esenciales antes de aplicar algoritmos basados en distancia.



Consideraciones Adicionales sobre la Distancia

Variables categóricas

Es importante considerar que las variables categóricas no pueden ser utilizadas directamente en los cálculos de distancia. Por este motivo, es necesario convertirlas previamente a un formato numérico mediante técnicas como One-Hot Encoding o variables dummy.

Maldición de la dimensionalidad

En problemas de alta dimensionalidad, el concepto de distancia puede perder significado debido a la **maldición de la dimensionalidad**. En estos casos, todas las distancias tienden a ser similares, dificultando la separación de clases o la detección de patrones. Por ello, es recomendable reducir la cantidad de características cuando sea posible.

Ejemplo Práctico de Cálculo de Distancia

Supongamos que tenemos dos observaciones representadas por sus edades y salarios:

- Persona A: Edad = 25, Salario = 30000
- Persona B: Edad = 40, Salario = 50000

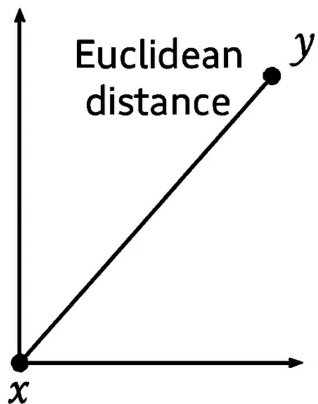
Distancia Euclideana:

$$Distance = \sqrt{(25 - 40)^2 + (30000 - 50000)^2}$$

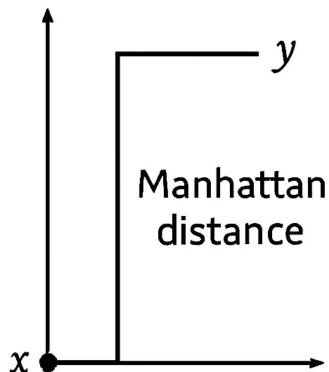
$$Distance = \sqrt{225 + 400000000} = \sqrt{400000225} \approx 20000.006$$

Como puedes observar, la distancia está fuertemente influenciada por la escala de la variable **Salario**. Por ello, en la práctica es recomendable **escalar los datos** antes de aplicar algoritmos que utilicen estas métricas.

Euclidean Distance



Manhattan distance



Tipos de Distancia: Manhattan y Euclidean

En el ámbito del aprendizaje automático, la elección de la métrica de distancia adecuada es esencial para obtener buenos resultados, especialmente en modelos basados en similitud. Existen diversas formas de medir la distancia entre dos puntos en un espacio n -dimensional, siendo las más comunes la distancia **Euclidean** y la distancia **Manhattan**.

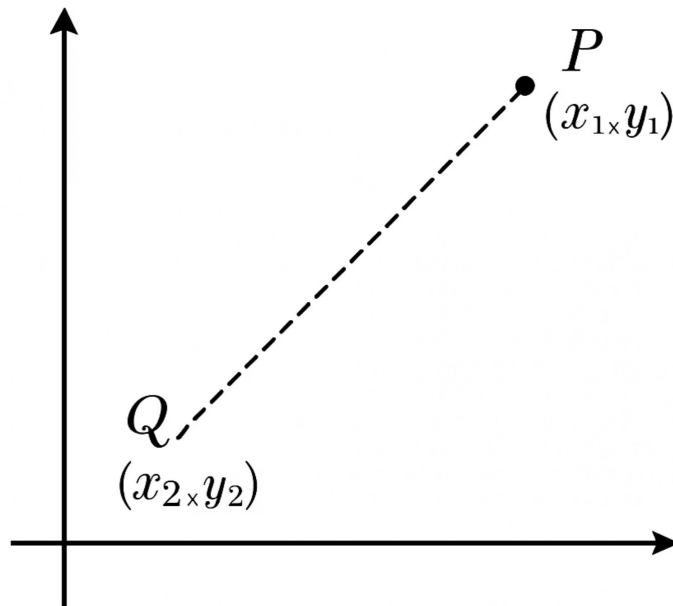
Distancia Euclideana

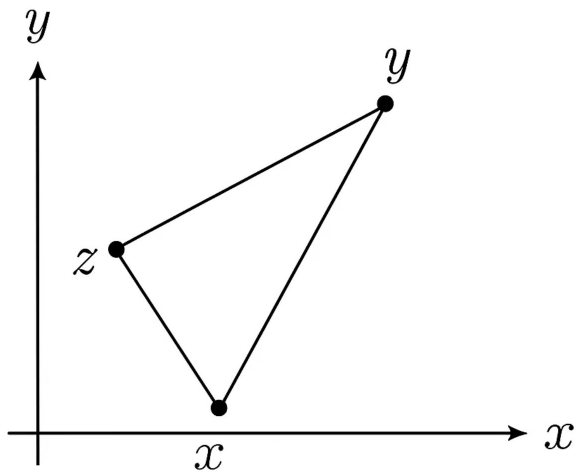
La **distancia Euclideana** es la más utilizada y representa la distancia "en línea recta" entre dos puntos.

Matemáticamente, se calcula como la raíz cuadrada de la suma de las diferencias al cuadrado entre las coordenadas de los puntos. Es decir, para dos puntos $A=(x_1,y_1)$ y $B=(x_2,y_2)$, la distancia Euclideana es:

$$d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$d = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$





1. $d(x, y) \geq 0$
2. $d(x, y) = d(y, x)$
3. $d(x, y) \leq d(x, z) + d(z, y)$

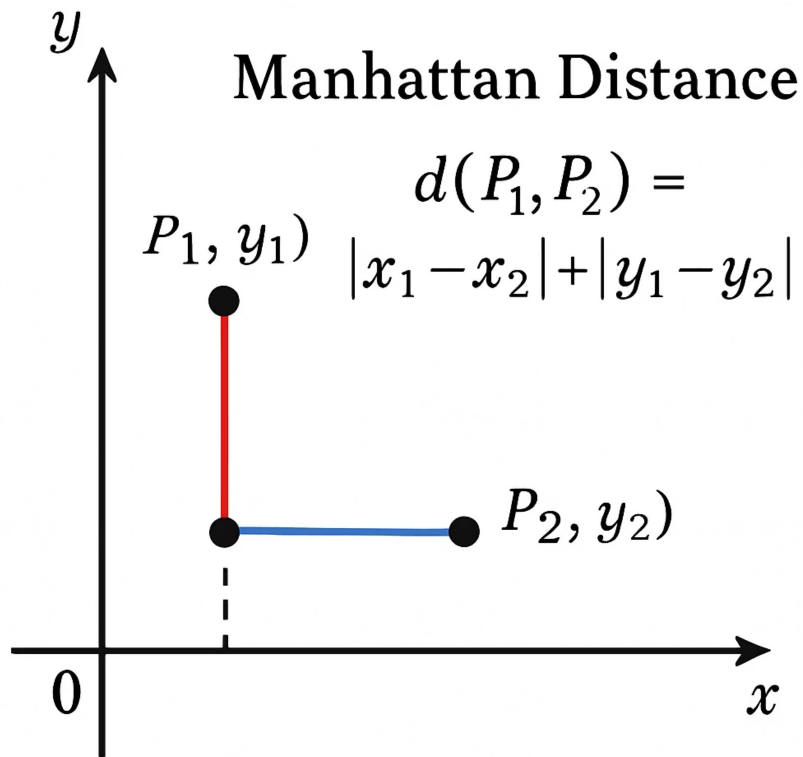
Características de la Distancia Euclídeana

Esta métrica es adecuada cuando todas las variables son continuas, están en la misma escala y la relación entre las características es lineal. Sin embargo, puede verse afectada por valores extremos y diferencias de escala entre las variables.

Distancia de Manhattan

Por otro lado, la **distancia de Manhattan**, también conocida como distancia del taxi o L1, mide la suma de las diferencias absolutas entre las coordenadas. Su fórmula es:

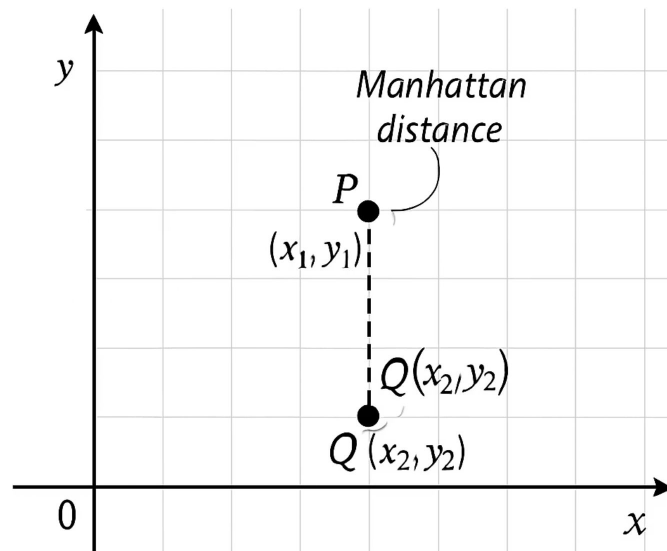
$$d(A, B) = |x_2 - x_1| + |y_2 - y_1|$$



Características de la Distancia de Manhattan

Esta métrica es apropiada cuando las variables son independientes y no existe una relación geométrica clara. Es menos sensible a los valores atípicos y, en algunos casos, puede ser más interpretativa, ya que refleja desplazamientos acumulados.

Manhattan Distance Properties



1. $D(P, Q) \geq 0$
2. $D(P, Q) = 0 \Leftrightarrow P = Q$
3. $D(P, Q) \leq D(P, R) + D(R, Q)$

Horizontal
/ Vertical



Comparación entre Métricas de Distancia

La principal diferencia entre ambas métricas radica en cómo consideran la distancia. La distancia Euclideana mide la separación directa entre dos puntos, mientras que la distancia de Manhattan mide la cantidad total de desplazamientos horizontales y verticales necesarios para llegar de un punto a otro.

La elección de una métrica u otra depende del tipo de datos y del problema a resolver. Por ejemplo, en problemas de clasificación utilizando K-Nearest Neighbors, la distancia Euclideana suele ser la opción por defecto, pero en casos con variables independientes y ruidosas, la distancia Manhattan puede ofrecer mejores resultados.

Ejemplo Práctico de Cálculo de Distancias

Supongamos que tenemos dos puntos en un espacio bidimensional:

- $A = (3, 4)$
- $B = (7, 1)$

Distancia Euclídeana:

$$d(A, B) = \sqrt{(7 - 3)^2 + (1 - 4)^2}$$

$$d(A, B) = \sqrt{16 + 9} = \sqrt{25} = 5$$

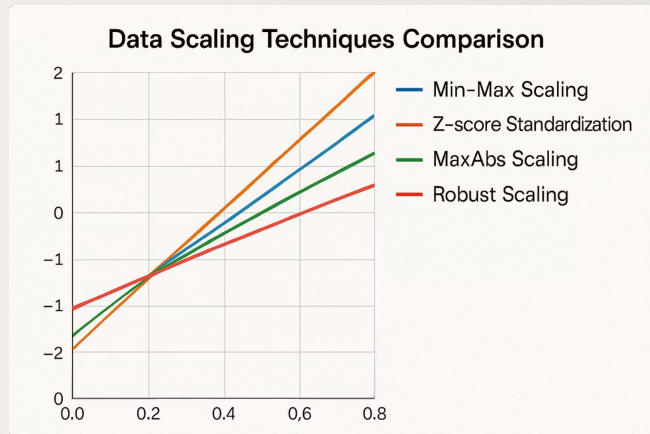
Distancia de Manhattan:

$$d(A, B) = |7 - 3| + |1 - 4| = 4 + 3 = 7$$

En este caso, la distancia Euclídeana es 5 y la distancia de Manhattan es 7. Ambas métricas miden la separación entre los puntos, pero bajo diferentes criterios geométricos. La elección de una u otra dependerá del contexto del análisis.

Técnicas Más Usadas de Escalamiento

El **escalamiento de datos** es una técnica esencial dentro del preprocesamiento, cuyo objetivo es poner todas las variables numéricas en un rango comparable. Muchas veces, los datos presentan variables con magnitudes muy distintas; por ejemplo, una variable puede representar edad en años y otra ingresos anuales en miles. Estas diferencias afectan el desempeño de los algoritmos de aprendizaje automático, especialmente aquellos que dependen de la medición de distancias o que son sensibles a la escala, como KNN, SVM o redes neuronales.



Normalización Min-Max

Una de las técnicas más utilizadas para el escalamiento es la **normalización Min-Max**. Este método transforma los valores de una variable para que estén dentro de un rango específico, comúnmente entre 0 y 1. La fórmula para aplicar Min-Max Scaling es:

$$X_{\text{escalado}} = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

Este tipo de escalamiento conserva la distribución original de los datos, pero puede verse afectado por la presencia de valores atípicos, ya que los extremos definen el rango de transformación. Por lo tanto, es recomendable aplicarlo cuando los datos no presentan outliers relevantes.

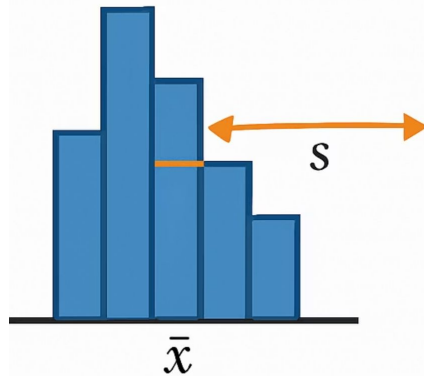
Estandarización (Z-Score)

Otra técnica muy común es la **estandarización**, también conocida como **Z-Score Normalization**. Este método transforma los datos para que tengan una media de 0 y una desviación estándar de 1. La fórmula utilizada es:

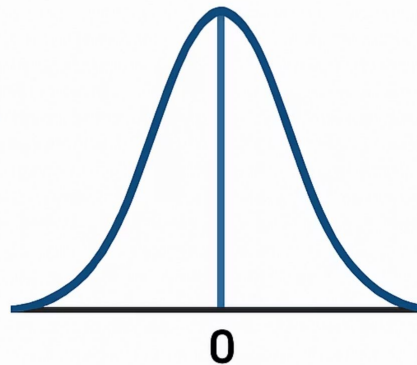
$$X_{\text{escalado}} = \frac{X - \mu}{\sigma}$$

donde μ es la media y σ la desviación estándar de la variable. La estandarización no limita los valores a un rango específico, pero es robusta ante la presencia de valores extremos y es adecuada cuando los datos siguen una distribución aproximadamente normal.

Z-Score Standardization



Histogram
of dataset



Standard normal
distribution

Elección de la Técnica de Escalamiento

Min-Max Scaling

- Ideal para datos con distribución desconocida
- Útil cuando se requiere un rango específico (0-1)
- Sensible a valores atípicos
- Preserva la forma original de la distribución

Estandarización

- Ideal para datos con distribución normal
- Robusta ante valores atípicos
- No limita a un rango específico
- Útil para algoritmos que asumen normalidad

Consideraciones Adicionales sobre Escalamiento

Algoritmos sensibles a la escala

La elección entre Min-Max y estandarización depende de la naturaleza de los datos y del algoritmo utilizado. Por ejemplo, los modelos basados en distancia, como KNN y K-Means, funcionan mejor cuando los datos están normalizados o estandarizados.

Algoritmos insensibles a la escala

Por otro lado, los modelos basados en árboles, como Random Forest, no requieren escalamiento, ya que son insensibles a la magnitud de las variables.

Variables categóricas

Es importante recordar que el escalamiento debe aplicarse únicamente a las variables numéricas. Las variables categóricas codificadas mediante One-Hot Encoding o variables dummy no deben ser escaladas, ya que su interpretación perdería sentido.

Implementación de Escalamiento con Scikit-Learn

La librería **Scikit-Learn** ofrece herramientas muy eficientes para llevar a cabo el escalamiento de datos, facilitando la implementación de técnicas como la normalización Min-Max y la estandarización. Estas herramientas permiten transformar las variables numéricas de manera rápida y sencilla, asegurando que estén en una escala adecuada para el entrenamiento de los modelos.

Conclusiones

Importancia del preprocesamiento

El preprocesamiento y el escalamiento de datos son pasos esenciales en cualquier flujo de trabajo de aprendizaje automático. Como hemos visto a lo largo de este manual, la calidad y la correcta transformación de los datos tienen un impacto directo en el rendimiento de los modelos, afectando su capacidad para aprender patrones y realizar predicciones precisas.

Técnicas fundamentales

Las técnicas de preprocesamiento permiten limpiar, transformar y preparar los datos para que sean interpretables por los algoritmos. Actividades como la codificación de variables categóricas mediante Label Encoding, One-Hot Encoding y variables dummy, así como la imputación de valores faltantes y la eliminación de duplicados, forman parte de este proceso indispensable.

Herramientas disponibles

Finalmente, la implementación de estas técnicas utilizando la librería **Scikit-Learn** facilita el desarrollo de proyectos de Machine Learning, permitiendo automatizar los flujos de trabajo y garantizar la reproducibilidad de los resultados. La combinación de teoría y práctica presentada en este manual proporciona una base sólida para enfrentar desafíos reales en la ciencia de datos.

Live Coding

¿En qué consistirá la Demo?

Vamos a construir un pipeline completo de preprocesamiento que maneje datos categóricos, valores nulos y escalamiento. Además, vamos a ver cómo cambia la distancia entre puntos antes y después del escalado.

1. Cargar un dataset de ejemplo
2. Separar columnas numéricas y categóricas
3. Definir imputadores y codificadores
4. Combinar todo en un ColumnTransformer
5. Enlazarlo a un Pipeline
6. Aplicar fit_transform al dataset
7. Calcular distancias entre observaciones con y sin escalado



Momento:

Time-out!



5 -10 min.





Ejercicio

Construcción de pipeline + análisis de distancias

Construcción de pipeline + análisis de distancias

Contexto: 🙌

Estás desarrollando un sistema de recomendación de productos basado en similitud entre usuarios. Necesitas asegurarte de que los datos estén correctamente procesados para que las distancias entre usuarios sean significativas.

Consigna: 📝

Construí un pipeline que:

1. Impute valores faltantes
2. Codifique variables categóricas
3. Escale variables numéricas
4. Luego, calcule las distancias Euclideana y Manhattan entre dos observaciones antes y después del pipeline.

Paso a paso: ⚙️

1. Genere y cargue el dataset usuarios.csv
2. Identifique columnas numéricas y categóricas
3. Define imputadores, codificadores y escaladores
4. Cree el ColumnTransformer
5. Cree el Pipeline completo
6. Aplique transformaciones al dataset
7. Elige dos observaciones y calcular las distancias entre ellas
8. Analice cómo cambian las distancias después del procesamiento

Tiempo 🕒: 45 Minutos

¿Alguna consulta?





Resumen

¿Qué logramos en esta clase?



- ✓ Aplicamos transformaciones avanzadas con `ColumnTransformer`
- ✓ Construimos pipelines de preprocesamiento reutilizables
- ✓ Calculamos y comparamos distancias entre observaciones
- ✓ Comprendimos la importancia del escalamiento en modelos basados en similitud



¡Ponte a prueba!

Momento de ejercitación

Te invitamos a aprovechar esta última sección del espacio sincrónico para realizar de manera individual las **actividades disponibles en la plataforma**. Estas propuestas son clave para afianzar lo trabajado y **forman parte obligatoria del recorrido de aprendizaje**.

👉 **Análisis de caso** ————— 👉 **Selección Múltiple**

👉 **Comprensión lectora**

Si al resolverlas surge alguna duda, compartela o tráela al próximo encuentro sincrónico.

< **¡Muchas gracias!** >

